

Machine Learning for Physicists

A Nutshell Guide to the Practical Man

Caio Laganá Fernandes

Preface

So, you're a physicist looking to dive into the world of machine learning. You have a deep understanding of calculus, are familiar with experimental data-fitting methods, and have mastered concepts like entropy and non-linearity—yet you have only a vague idea of how a machine learning algorithm is actually written and how it works? Then this book is for you. And the good news is: your learning curve will grow fast. I've been in your shoes. I used to view machine learning as a distant horizon, but I soon realized that the knowledge I'd gained in physics made it incredibly easy to quickly master the concepts and engineering behind the field. So, come along and be amazed.

Chapter 1

One-neuron ML

You have been using a single-neuron network since the first time you fitted experimental data, but perhaps no one ever told you that. Let's recall the basics. You have a set of N experimental data points

$$(x_i, y_i) \tag{1.1}$$

through which to fit a linear function

$$f(x) = ax + b. \tag{1.2}$$

The Mean Squared Error Loss, defined as

$$\begin{aligned} L_{\text{MSE}}(a, b) &= \frac{1}{N} \sum_{i=1}^N [y_i - f(x_i)]^2 \\ &= \frac{1}{N} \sum_{i=1}^N [y_i - (ax_i + b)]^2 \end{aligned} \tag{1.3}$$

tells us how distant the curve $f(x)$ is from the datapoints. Fitting $f(x)$ to the datapoints (x_i, y_i) means minimizing $L_{\text{MSE}}(a, b)$ with respect to a and b , that is,

$$\frac{\partial}{\partial(a, b)} L_{\text{MSE}}(a, b) = 0. \tag{1.4}$$

No surprises so far, right? Well, what if I tell you the above is the simplest possible neural network? Indeed, it is a single-neuron network, where (1.2) are the model's weights, (1.3) is the loss function and (1.4) is the embryo of the gradient descent method.

Solving (1.4) for a and b , which, in this simple case is possible analytically, fixes the parameters to a_0 and b_0

$$\begin{aligned} a_0 &= \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2} \\ b_0 &= \bar{y} - a_0 \bar{x} \end{aligned} \tag{1.5}$$

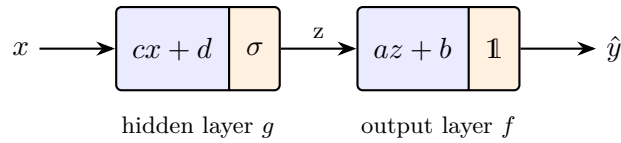
and we end up with a model $f_0(x) = a_0x + b_0$ (in (1.5), $\bar{x}$ and $\bar{y}$ are the mean values of the x_i and y_i datapoints). This model is a single neuron, no hidden layers: it receives one input parameter and outputs a number. f_0 has learned from data (1.1) through (1.4) and can now predict on any given x . Pretty cool, huh? The concepts were sitting there all the time. Take a moment to re-read the definitions and let's jump to the second simplest model: a two-neuron network.

Chapter 2

Two-neuron ML

Our neuron from Chap. 1 was of the form $f(x) = ax + b$ (always think of it as receiving one value x and, upon training for a, b parameters, outputting another value $a_0x + b_0$). Let us now place a second neuron, g , before it, so that g receives the input and f produces the final output. This new neuron will be of the form $g(x) = \sigma(cx + d)$, where $\sigma(x) = 1/(1 + e^{-x})$. The need for σ will be elucidated later; for now, just accept it as a magical ingredient. In fact, it is the neuron's so-called activation function!

Placing one neuron before another means composing the two functions, that is, given input x , the network outputs $f(g(x))$. In other words, input x feeds a (hidden) layer g , which applies its weight c and bias d , activates through σ and feeds output layer f , which in turn applies weight a and bias b and outputs $\hat{y}$. Layer f has identity operator $\mathbf{1}$ as its activation function. A visual diagram may help.



Our task now is to optimize the network for parameters a, b, c, d over training data. The loss function is similar to (1.3), just generalized for the two-neuron case,

$$L_{\text{MSE}}(a, b, c, d) = \frac{1}{N} \sum_{i=1}^N [y_i - f(g(x_i))]^2. \quad (2.1)$$

This time, however, MSE minimization, stated as

$$\frac{\partial}{\partial(a, b, c, d)} L_{\text{MSE}}(a, b, c, d) = 0 \quad (2.2)$$

cannot be solved analytically as in (1.5), and numerical methods must be used. The recipe is as follows: start at a certain initial position (a, b, c, d) in the four-dimensional parameter space,

then slightly change each coordinate by an amount according to¹:

$$\begin{aligned} a &\rightarrow a - \alpha \frac{\partial L_{\text{MSE}}}{\partial a} \\ b &\rightarrow b - \alpha \frac{\partial L_{\text{MSE}}}{\partial b} \\ c &\rightarrow c - \alpha \frac{\partial L_{\text{MSE}}}{\partial c} \\ d &\rightarrow d - \alpha \frac{\partial L_{\text{MSE}}}{\partial d} \end{aligned} \quad (2.3)$$

The term α is called the learning rate (hallelujah, a name you’ve heard before, now in a clear context!). The derivatives of L_{MSE} in (2.3) point to the max uphill on the hypersurface, so the minus sign will move the initial coordinate to a new position where L_{MSE} is lower. After repeating this procedure a couple of times, (2.2) will be solved to a certain desired precision (yes, I am oversimplifying, this step is a billion-dollar industry).

The recipe just described for adjusting (a, b, c, d) parameters based on the partial derivatives (2.3) is called *gradient descent*. The computation of the derivatives themselves will lead to the concept of *backpropagation*. Stated like this, with bare partial derivatives and almost no intuition of what is going on behind the scenes, the recipe sounds a bit barren. Let me improve it.

The central idea of gradient descent and backpropagation is to tell (or, more precisely, to *signal*) the neurons in which direction they must adjust their parameters, and by how much, in order to minimize the final loss. Think about it: how can a hidden neuron (in our two-neuron model, g) possibly gather information about the loss at the very output? This is the central problem to be addressed. Let’s dive a little into the algebra.

The partial derivatives present in (2.3), obtained through chain-rule derivatives on (2.1), can, in this case, be taken explicitly:

$$\begin{aligned} \frac{\partial L}{\partial a} &= \frac{\partial L}{\partial f} \cdot \frac{\partial f}{\partial a} = -\frac{2}{N} \sum_i [y_i - f(g(x_i))] g(x_i) \\ \frac{\partial L}{\partial b} &= \frac{\partial L}{\partial f} \cdot \frac{\partial f}{\partial b} = -\frac{2}{N} \sum_i [y_i - f(g(x_i))] \\ \frac{\partial L}{\partial c} &= \frac{\partial L}{\partial f} \cdot \frac{\partial f}{\partial g} \cdot \frac{\partial g}{\partial c} = -\frac{2a}{N} \sum_i [y_i - f(g(x_i))] \sigma'(cx_i + d) x_i \\ \frac{\partial L}{\partial d} &= \frac{\partial L}{\partial f} \cdot \frac{\partial f}{\partial g} \cdot \frac{\partial g}{\partial d} = -\frac{2a}{N} \sum_i [y_i - f(g(x_i))] \sigma'(cx_i + d) \end{aligned} \quad (2.4)$$

A suitable reparametrization of variables will be of great algebraic assistance. (We know from other branches of physics—electromagnetism, condensed matter, particle physics—that a “suitable change of variables” often carries deep phenomenological significance. So, hold on!)

$$\begin{array}{ll} x_i & \text{input} \\ u_i = cx_i + d & \text{hidden pre-activation} \\ z_i = \sigma(u_i) & \text{hidden activation} \\ \hat{y}_i = f(g(x_i)) = az_i + b & \text{network output} \end{array} \quad (2.5)$$

In terms of (2.5), define the *error signal* as

$$\delta_i = \frac{\partial L}{\partial \hat{y}_i} = -\frac{2}{N} (y_i - \hat{y}_i), \quad (2.6)$$

¹Picture this process as finding a minimum on a graph. For reference, in Sec. ?? the graph of the loss function, $(a, b, L_{\text{MSE}}(a, b))$, was a two-dimensional paraboloid embedded in $\mathbb{R}^3$; in this two-neuron scenario, the graph is a four-dimensional hypersurface embedded in $\mathbb{R}^5$. The paraboloid has a clear, well-defined minimum; this time, there may be several local minima plus saddle points and plateaus, and which one you land in depends on the starting point.

and the *hidden error signal* as

$$\delta_i^h = \delta_i a \sigma'(u_i). \quad (2.7)$$

As the name suggests, the error signals will *signal* the neurons in which direction to change their parameters, and by how much. With all these new variables, the partials (2.4) read

$$\begin{aligned} \frac{\partial L}{\partial a} &= \sum_i \delta_i z_i \\ \frac{\partial L}{\partial b} &= \sum_i \delta_i \\ \frac{\partial L}{\partial c} &= \sum_i \delta_i^h x_i \\ \frac{\partial L}{\partial d} &= \sum_i \delta_i^h \end{aligned} \quad (2.8)$$

Phew! Much cleaner than (2.4)! And in return we've gained an astonishing behaviour: instead of computing all the messy partial derivatives in (2.4), all we need to do in order to calibrate each neuron's parameters is to compute its respective error signal and apply elementary algebra. This shortcut is called *backpropagation*. The name stems from the fact that, starting at the output and going backwards, each crossed layer will render the neuron one error signal. Look carefully: a and b each gained a δ_i plus z_i for a ; similarly, c and d each gained a δ_i^h plus x_i for c . Were there a third layer, the recipe would repeat.

This behaviour showed up explicitly in this two-neuron toy model, but in fact it generalizes to networks of any size. This prepares our terrain for the next chapter.

Chapter 3

Multi-neuron ML

We now have all necessary ingredients to build a multi-neuron network. So, let's do it! The first step is to adapt the input layer to be able to receive, instead of a single scalar x , a more generic object. You may argue “the most generic object is an $N \times M$ matrix, or a tensor of arbitrary rank!” And you're right, as I thought I was the first time I philosophized about the subject. However... When you stop to think about it, you realize that any generic object, whether it is a matrix or a tensor, can be *flattened* to a vector of some dimension. This flattening process can always take place outside of the network, in a data preprocessing stage. So, as far as the network itself is concerned, the most general input object is a N -dimensional vector.

The above considerations lead us to define the architecture of the first layer as an N -neuron input layer. I will make use of tensor index notation (as in General Relativity books) to write the input parameter of (2.5) as

$$x_\mu^i \quad \text{input} \quad (3.1)$$

I have promoted the i -index to the top (remember, this is the index labeling the i th data sample) while using greek letters at the bottom as the vector indexes. With this notation, the hidden pre-activation step reads

$$u_\mu^i = w_{\mu\nu} x_\nu^i + b_\mu \quad \text{hidden pre-activation} \quad (3.2)$$

In (3.2), the weight c from (2.5) has turned into the $w_{\mu\nu}$ matrix (the letter w is the standard one for referring to the weights; I've been using a , b , c and d for laziness) whereas the bias parameter, called d in (2.5), has turned into a bias vector b_μ . Very important: the ν -index range from 1 to the dimension of the input vector; the μ index range from 1 to the size of the hidden layer. The $w_{\mu\nu}$ matrix connects every input scalar to every neuron in the hidden layer.

One last step to notice before we conclude the architecture of the input and hidden layers: the activation function for the neurons in the hidden layer must be chosen.

$$z_\mu^i = \sigma(u_\mu^i) \quad \text{hidden activation} \quad (3.3)$$

There are a few options for σ in the market, all listed in Appendix A. The standard choice for hidden layers is ReLU, though it may change from problem to problem.

That said, we can move on to the final step,: defining the output layer. This is where the nature of the problem comes into play.

$$y_\eta^i = w_{\eta\mu} z_\mu^i + b_\eta \quad \text{network output} \quad (3.4)$$

Chapter 4

Appendix A

- *ReLU* (Rectified Linear Unit), $\text{ReLU}(x) = \max(0, x)$ — the default choice: trivially cheap to evaluate and it never saturates for $x > 0$, so the error signal survives many layers.
- *Leaky ReLU*, $\text{LReLU}(x) = \max(\gamma x, x)$ with a small fixed γ (typically 0.01) — cures the “dead neuron” pathology of ReLU, whose derivative vanishes identically for negative inputs.
- *PReLU* (Parametric ReLU), same form as Leaky ReLU but with γ promoted to a trainable parameter — lets the network learn how leaky it wants to be, at the cost of one extra parameter per neuron (or per layer).
- *ELU* (Exponential Linear Unit),

$$\text{ELU}(x) = \begin{cases} x, & x > 0 \\ \beta(e^x - 1), & x \leq 0 \end{cases}$$

— smooth everywhere and saturating at $-\beta$ for large negative inputs, which pulls the mean activation toward zero and speeds up convergence.

- *GELU* (Gaussian Error Linear Unit), $\text{GELU}(x) = x \Phi(x) = \frac{x}{2} \left[1 + \text{erf}\left(\frac{x}{\sqrt{2}}\right) \right]$, with Φ the standard normal cumulative distribution — a smooth, probabilistically motivated gate that weights each input by how likely it is to be positive; the standard in modern transformer architectures.
- *Sigmoid/Logistic*, $\sigma(x) = 1/(1 + e^{-x})$ — our old friend from chapter 2, squashing the real line into $(0, 1)$; ideal for probabilities at the output, but its derivative never exceeds $1/4$, which starves deep networks of gradient.
- *Tanh*, $\tanh(x) = (e^x - e^{-x})/(e^x + e^{-x})$ — a rescaled sigmoid mapping onto $(-1, 1)$, preferable to it because it is centred at zero, though it saturates just as badly.